

# ECE 356 Lab 4

John Brisbois V00957875

Github link

<https://github.com/johnBrisbois02/ECE-356-Lab-4>

## Note

All code was written in c on a WSL running ubuntu, it was compiled and run using the gcc compiler in bash

example for q1 code

compile: gcc Lab4/Lab4-P2.c -o Lab4/Lab4-P1

then run: ./Lab4/Lab4-P1 (Exclude Lab4/ if current directory is where the code is stored)

## Question 1

After compiling and running this code, it will prompt you for the Scenario

Type 1 for deadlock, 2 for starvation, 3 for Proper resource management

Then hit enter for the program to start running to get the thread count prompt

Enter the number of threads (Min 1, Max 10)

Hitting anything other than 1 2 or 3 for scenario or above 10 or below 1 will tell you to reinput.

```
john@Computer:~/ece 356$ ./Lab4/Lab4-P1
Enter Scenario
1 for deadlock, 2 for starvation, 3 for Proper resource managment
1
Enter number of Threads (Max:10 Min:1)
10
```

Example Input Prompt

### Phase 1 (Deadlock)

For the deadlock to occur there needs to be at least 4 threads (3 to hold the runway 1 to hold control tower) less than that and there are not enough threads to cause a deadlock.

With no deadlock there are enough times for ~20 takeoffs (possibly 19 depending on PC speed)

```

john@Computer:~/ece 356$ gcc Lab4/Lab4-P1.c -o Lab4/Lab4-P1
john@Computer:~/ece 356$ ./Lab4/Lab4-P1
Enter Scenario
1 for deadlock, 2 for starvation, 3 for Proper resource management
1
Enter number of Threads (Max:10 Min:1)
4
Thread[1] Aquired a runway
Thread[2] Aquired a runway
Thread[3] Aquired a runway
Thread[4] Aquired Control tower permission
Thread[1] Total takeoff count:0
Thread[2] Total takeoff count:0
Thread[3] Total takeoff count:0
Thread[4] Total takeoff count:0
Takeoff count = 0

```

#### Example Response 1 (Deadlock Occurring)

If there are less than 4 threads deadlock will likely not occur, but starvation will occur.

#### Phase 2 (Starvation)

With Phase 2 the deadlock is removed by removing the circular wait but there is nothing that prevents Starvation from occurring

There is a 0.5 second wait after the thread locks and before it unlocks, removing this wait can cause starvation but it ultimately is random, I added the wait, so the starvation becomes more apparent.

```

john@Computer:~/ece 356$ ./Lab4/Lab4-P1
Enter Scenario
1 for deadlock, 2 for starvation, 3 for Proper resource management
2
Enter number of Threads (Max:10 Min:1)
10
Thread[1] Aquired Control tower permission
Thread[1] Aquired a runway
Thread[1] Taking off
Thread[1] Aquired Control tower permission
Thread[1] Aquired a runway
Thread[1] Taking off
Thread[1] Aquired Control tower permission
Thread[1] Aquired a runway
Thread[1] Taking off
Thread[1] Aquired Control tower permission
Thread[1] Aquired a runway
Thread[1] Taking off
Thread[1] Aquired Control tower permission
Thread[1] Aquired a runway
Thread[1] Taking off
Thread[1] Aquired Control tower permission

```

```

Thread[1] Aquired Control tower permission
Thread[1] Aquired a runway
Thread[1] Taking off
Thread[1] Total takeoff count:20
Thread[2] Total takeoff count:0
Thread[3] Total takeoff count:0
Thread[4] Total takeoff count:0
Thread[5] Total takeoff count:0
Thread[6] Total takeoff count:0
Thread[7] Total takeoff count:0
Thread[8] Total takeoff count:0
Thread[9] Total takeoff count:0
Thread[10] Total takeoff count:0
Takeoff count = 20
john@Computer:~/ece 356$ █

```

#### Example Response 2 (Top and bottom of starvation output)

### Phase 3 (Proper Functionality)

To implement phase 3, I introduced a FIFO resource queue, so every thread got an equal number of takeoffs, after each thread completes a takeoff, they are removed from the top and put onto the bottom of the queue.

```
john@Computer:~/ece 356$ ./Lab4/Lab4-P1
Enter Scenario
1 for deadlock, 2 for starvation, 3 for Proper resource managment
3
Enter number of Threads (Max:10 Min:1)
10
Thread[1] Aquired Control tower permission
Thread[1] Aquired a runway
Thread[1] Taking off
Thread[2] Aquired Control tower permission
Thread[2] Aquired a runway
Thread[2] Taking off
Thread[3] Aquired Control tower permission
Thread[3] Aquired a runway
Thread[3] Taking off
Thread[9] Aquired Control tower permission
Thread[9] Aquired a runway
Thread[9] Taking off
Thread[10] Aquired Control tower permission
Thread[10] Aquired a runway
Thread[10] Taking off
Thread[1] Total takeoff count:2
Thread[2] Total takeoff count:2
Thread[3] Total takeoff count:2
Thread[4] Total takeoff count:2
Thread[5] Total takeoff count:2
Thread[6] Total takeoff count:2
Thread[7] Total takeoff count:2
Thread[8] Total takeoff count:2
Thread[9] Total takeoff count:2
Thread[10] Total takeoff count:2
Takeoff count = 20
john@Computer:~/ece 356$
```

Example Response 3 (Top and bottom of FIFO Queue output)

## Question 2

This code runs for a set amount of time so for deadlocks it will sit with nothing occurring and when the threads are able to operate normally, they will finish before the main script is done waiting.

The script will terminate in 10 seconds after threads are called or will be trapped waiting for the deadlocked threads.

Like the last Part this code asks for which Scenario you want and how many threads.

```
john@Computer:~/ece 356$ gcc Lab4/Lab4-P2.c -o Lab4/Lab4-P2
john@Computer:~/ece 356$ ./Lab4/Lab4-P2
Input the Scenario
1 for Naive Deadlock, 2 for 2PL, 3 for Deadlock detection, 4 for Bankers
2
Input Thread Count (1 Min, 10 Max)
4
```

Example Input Request

## Phase 1(Naïve Deadlock)

Selecting 1 with more than one array will likely result in a deadlock. The code waits for 10s then tries to terminate the threads, but it is unable to due to the deadlock.

```
john@Computer:~/ece 356$ gcc Lab4/Lab4-P2.c -o Lab4/Lab4-P2
john@Computer:~/ece 356$ ./Lab4/Lab4-P2
Input the Scenario
1 for Naive Deadlock, 2 for 2PL, 3 for Deadlock detection, 4 for Bankers
1
Input Thread Count (1 Min, 10 Max)
4
Initial Account Values
Account ID: A, Balance: 1000
Account ID: B, Balance: 500
Account ID: C, Balance: 750
Account ID: D, Balance: 1200
Thread[1] Attempting to lock account A
Thread[1] Succesfully Locked account A
Thread[2] Attempting to lock account B
Thread[2] Succesfully Locked account B
Thread[3] Attempting to lock account C
Thread[3] Succesfully Locked account C
Thread[4] Attempting to lock account D
Thread[4] Succesfully Locked account D
Thread[1] Attempting to lock account B
Thread[3] Attempting to lock account D
Thread[2] Attempting to lock account C
Thread[4] Attempting to lock account A
Final Account Values
Account ID: A, Balance: 1000
Account ID: B, Balance: 500
Account ID: C, Balance: 750
Account ID: D, Balance: 1200
```

Example Response (Balance remains unchanged due to deadlock as well as inability to terminate threads)

## Phase 2

Selecting 2 for the input will prevent deadlock due since circular there is no more circular waiting.

Although in a process that needs to lock and unlock many times it may lead to starvation in this case there is no starvation since there are a low number of tasks being completed.

```

john@Computer:~/ece 356$ gcc Lab4/Lab4-P2.c -o Lab4/Lab4-P2
john@Computer:~/ece 356$ ./Lab4/Lab4-P2
Input the Scenario
1 for Naive Deadlock, 2 for 2PL, 3 for Deadlock detection, 4 for Bankers
2
Input Thread Count (1 Min, 10 Max)
4
Initial Account Values
Account ID: A, Balance: 1000
Account ID: B, Balance: 500
Account ID: C, Balance: 750
Account ID: D, Balance: 1200
Thread[1] Attempting to lock account A
Thread[1] Succesfully Locked account A
Thread[2] Attempting to lock account B
Thread[2] Succesfully Locked account B
Thread[3] Attempting to lock account C
Thread[3] Succesfully Locked account C
Thread[4] Attempting to lock account A
Thread[1] Attempting to lock account B
Thread[2] Attempting to lock account C
Thread[3] Attempting to lock account D
Thread[3] locked accounts C and D Transferring now
Thread[3] Transfer Complete unlocking accounts C and D
Thread[2] locked accounts B and C Transferring now
Thread[2] Transfer Complete unlocking accounts B and C
Thread[1] locked accounts A and B Transferring now
Thread[1] Transfer Complete unlocking accounts A and B
Thread[4] Succesfully Locked account A
Thread[4] Attempting to lock account D
Thread[4] locked accounts A and D Transferring now
Thread[4] Transfer Complete unlocking accounts A and D
Final Account Values
Account ID: A, Balance: 1250
Account ID: B, Balance: 300
Account ID: C, Balance: 800
Account ID: D, Balance: 1050
john@Computer:~/ece 356$

```

Example response with correct output values.

### Phase 3 (Deadlock Detection)

The deadlock detection runs before any of the threads run and detects if there is a possibility for a deadlock to occur, if it detects one it will remove the thread at the start of the loop (thread for account A here). Ex. T1->T2->T3->T4->T1 Detects possible deadlock and removes T1.

This repeats until there are no more deadlock possibilities.

```

john@Computer:~/ece 356$ ./Lab4/Lab4-P2
Input the Scenario
1 for Naive Deadlock, 2 for 2PL, 3 for Deadlock detection, 4 for Bankers
3
Input Thread Count (1 Min, 10 Max)
4
Initial Account Values
Account ID: A, Balance: 1000
Account ID: B, Balance: 500
Account ID: C, Balance: 750
Account ID: D, Balance: 1200
Cycle found terminating Thread[1]
Thread[2] Attempting to lock account B
Thread[2] Succesfully Locked account B
Thread[3] Attempting to lock account C
Thread[3] Succesfully Locked account C
Thread[4] Attempting to lock account D
Thread[4] Succesfully Locked account D
Thread[2] Attempting to lock account C
Thread[3] Attempting to lock account D
Thread[4] Attempting to lock account A
Thread[4] locked accounts A and A Performing transaction now
Thread[4] Transfer Complete unlocking accounts D and A
Thread[3] locked accounts D and D Performing transaction now
Thread[3] Transfer Complete unlocking accounts C and D
Thread[2] locked accounts C and C Performing transaction now
Thread[2] Transfer Complete unlocking accounts B and C
Final Account Values
Account ID: A, Balance: 1050
Account ID: B, Balance: 500
Account ID: C, Balance: 800
Account ID: D, Balance: 1050
john@Computer:~/ece 356$ █

```

Example Response (Thread 1 terminated so A no longer receives the 200 transfers from B)

#### Phase 4 (Bankers)

Selecting this Scenario will make the threads get permission from the 'Banker' before they are allowed to try to access the locks. Before a thread tries to get a lock, it asks the banker, and the banker checks to see if allowing it to take all the resources it needs will stop another thread. If both locks that the thread will need are free it will allow it through, otherwise it will stop until both locks are available.

```

john@Computer:~/ece 356$ gcc Lab4/Lab4-P2.c -o Lab4/Lab4-P2
john@Computer:~/ece 356$ ./Lab4/Lab4-P2
Input the Scenario
1 for Naive Deadlock, 2 for 2PL, 3 for Deadlock detection, 4 for Bankers
4
Input Thread Count (1 Min, 10 Max)
4
Initial Account Values
Account ID: A, Balance: 1000
Account ID: B, Balance: 500
Account ID: C, Balance: 750
Account ID: D, Balance: 1200
Thread[1] got Banker's permission
Thread[1] Attempting to lock account A
Thread[1] Successfully Locked account A
Thread[1] Attempting to lock account B
Thread[1] locked accounts A and B Performing Transaction
Thread[3] got Banker's permission
Thread[3] Attempting to lock account C
Thread[3] Successfully Locked account C
Thread[3] Attempting to lock account D
Thread[3] locked accounts C and D Performing Transaction
Thread[1] Transaction Complete unlocking accounts A and B
Thread[1] returning locks to Banker
Thread[3] Transaction Complete unlocking accounts C and D
Thread[3] returning locks to Banker
Thread[4] got Banker's permission
Thread[4] Attempting to lock account A
Thread[4] Successfully Locked account A
Thread[4] Attempting to lock account D
Thread[4] locked accounts A and D Performing Transaction
Thread[2] got Banker's permission
Thread[2] Attempting to lock account B
Thread[2] Successfully Locked account B
Thread[2] Attempting to lock account C
Thread[2] locked accounts B and C Performing Transaction
Thread[4] Transaction Complete unlocking accounts A and D
Thread[4] returning locks to Banker
Thread[2] Transaction Complete unlocking accounts B and C
Thread[2] returning locks to Banker
Final Account Values
Account ID: A, Balance: 1250
Account ID: B, Balance: 300
Account ID: C, Balance: 800
Account ID: D, Balance: 1050
john@Computer:~/ece 356$ 

```

### Example Response

In this case where 1 got permission from the banker first (race condition) this means that it guarantees that 3 is next since it does not share any locks with thread 1. This becomes more apparent with more threads.

```
Thread[1] got Banker's permission
Thread[1] Attempting to lock account A
Thread[1] Successfully Locked account A
Thread[1] Attempting to lock account B
Thread[1] locked accounts A and B Performing Transaction
Thread[3] got Banker's permission
Thread[3] Attempting to lock account C
Thread[3] Successfully Locked account C
Thread[3] Attempting to lock account D
Thread[3] locked accounts C and D Performing Transaction
Thread[5] got Banker's permission
Thread[5] Attempting to lock account E
Thread[5] Successfully Locked account E
Thread[5] Attempting to lock account F
Thread[5] locked accounts E and F Performing Transaction
Thread[7] got Banker's permission
Thread[7] Attempting to lock account G
Thread[7] Successfully Locked account G
Thread[7] Attempting to lock account H
Thread[7] locked accounts G and H Performing Transaction
Thread[9] got Banker's permission
Thread[9] Attempting to lock account I
Thread[9] Successfully Locked account I
Thread[9] Attempting to lock account J
Thread[9] locked accounts I and J Performing Transaction
Thread[1] Transaction Complete unlocking accounts A and B
Thread[1] returning locks to Banker
Thread[3] Transaction Complete unlocking accounts C and D
Thread[3] returning locks to Banker
Thread[5] Transaction Complete unlocking accounts E and F
Thread[5] returning locks to Banker
Thread[4] got Banker's permission
Thread[4] Attempting to lock account D
Thread[4] Successfully Locked account D
Thread[4] Attempting to lock account E
```

Part of 10 Thread Bankers Response.

Its clear to see that Banker algorithm allows concurrency if there is no conflict of resources.